

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	<i>PRUEBAS DE SOFTWARE</i>				
TÍTULO DE LA PRÁCTICA:	<i>Desarrollo Orientado a Pruebas (TDD) y Comportamiento (BDD)</i>				
NÚMERO DE PRÁCTICA:	<i>02</i>	AÑO LECTIVO:	<i>2026</i>	NRO. SEMESTRE:	<i>VII</i>
FECHA DE PRESENTACIÓN	<i>29/04/2026</i>	HORA DE PRESENTACIÓN	<i>23:59</i>		
INTEGRANTE (s): Quispe Flores Marco Ramiro Quispe Madariaga Jeferson Jofre Ramirez Ccahuana Max Edu Valdiviezo Tovar Alexander				NOTA:	
DOCENTE(s): Prof. Robert Arisaca					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS 1. TDD: Validador de Identificadores (Basado en Myers) - Desarrolle un módulo que valide si el nombre de una variable (identificador) es válido según las reglas clásicas de Fortran mencionadas por Myers: - Debe tener entre 1 y 6 caracteres. - El primer carácter debe ser obligatoriamente una letra. - Los caracteres restantes pueden ser letras o dígitos. - Aplicar TDD para implementar las reglas una por una. Se deben diseñar casos de prueba para los valores límite (longitud 0, 1, 6 y 7) y para la robustez (caracteres especiales). - Entregables: <code>identificador.py</code> y <code>test_identificador.py</code>. Paso A: Red (Escribir una prueba que falle): Se escribe el test con los siguientes casos de prueba el cual fallara porque la función “<code>es_identificador_valido</code>” aún no existe.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

Python

```
import unittest
from identificador import es_identificador_valido

class TestIdentificador(unittest.TestCase):

    #tener entre 1 y 6 caracteres
    #el primer caracter debe ser una letra
    #los demas caracteres pueden ser letras o numeros

    def test_escenario_valido(self):

        self.assertTrue(es_identificador_valido("abc123"))
        self.assertTrue(es_identificador_valido("a1"))

    def test_escenario_longitud(self): #valores entre limites [1, 6]
        self.assertFalse(es_identificador_valido("")) # 0
        self.assertTrue(es_identificador_valido("a")) # 1
        self.assertTrue(es_identificador_valido("abcdef")) # 6
        self.assertFalse(es_identificador_valido("abcdefg")) # 7

    def test_escenario_primer_caracter_como_letra(self): #primer caracter como letra
        self.assertFalse(es_identificador_valido("1abc")) # 1

    def test_escenario_caracteres_invalidos(self): #caracteres restantes pueden ser letras o numeros
        self.assertFalse(es_identificador_valido("ab$12")) # $
        self.assertTrue(es_identificador_valido("abc12")) # caracteres validos

unittest.main()
```

Paso B: Green (Código mínimo para pasar):

Python

```
def es_identificador_valido(nombre):
    return len(nombre) >= 1 and len(nombre) <= 6
```

Las pruebas pasan 4 y fallan 2 los que son del primer caracter como letra y caracteres no válidos

```
F.F.
=====
FAIL: test_escenario_caracteres_invalidos (_main_.TestIdentificador.test_escenario_caracteres_invalidos)
-----
Traceback (most recent call last):
  File "c:\Users\jofre\OneDrive\Desktop\Lab\1.- Pruebas_de_software\lab02\test_identificador.py", line 25, in test_escenario_caracteres_invalidos
    self.assertFalse(es_identificador_valido("ab$12")) # $
    ~~~~~^~~~~~
AssertionError: True is not false

=====
FAIL: test_escenario_primer_caracter_como_letra (_main_.TestIdentificador.test_escenario_primer_caracter_como_letra)
-----
Traceback (most recent call last):
  File "c:\Users\jofre\OneDrive\Desktop\Lab\1.- Pruebas_de_software\lab02\test_identificador.py", line 22, in test_escenario_primer_caracter_como_letra
    self.assertFalse(es_identificador_valido("1abc")) # 1
    ~~~~~^~~~~~
AssertionError: True is not false

=====
Ran 4 tests in 0.002s

FAILED (failures=2)
PS C:\Users\jofre\OneDrive\Desktop\Lab\1.- Pruebas_de_software\lab02>
```

Paso C: Refactor y Nuevos Requisitos: Se implementa la lógica completa para cumplir con los 3 requisitos que son:

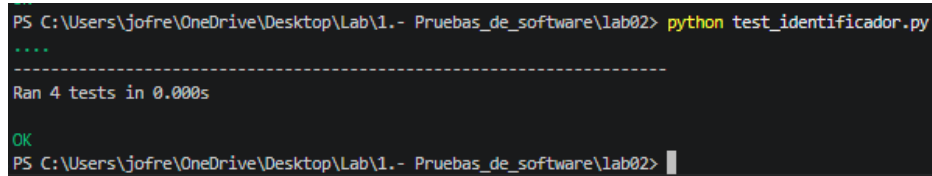
- De 1 a 6 caracteres
- Que el primer carácter sea una letra

Que el resto pueda ser caracteres letras o dígitos

Python

```
def es_identificador_valido(nombre):  
    if not isinstance(nombre, str): # Sea cadena de caracteres  
        return False  
  
    if len(nombre) < 1 or len(nombre) > 6: # caracteres [1,6]  
        return False  
  
    if not nombre[0].isalpha(): #primero como letra  
        return False  
  
    for c in nombre: #caracteres letras o numeros  
        if not c.isalnum():  
            return False  
  
    return True
```

Validación de Resultados: Ejecución de las pruebas en la terminal de VS Code: python test_identificador.py



```
PS C:\Users\jofre\OneDrive\Desktop\Lab\1.- Pruebas_de_software\lab02> python test_identificador.py  
.....  
-----  
Ran 4 tests in 0.000s  
OK  
PS C:\Users\jofre\OneDrive\Desktop\Lab\1.- Pruebas_de_software\lab02>
```

Todas las pruebas pasaron correctamente, se aplicaron los siguientes casos de prueba:

- Clases de equivalencia válida: "abc123", "a1" (Cumple con longitud, primer carácter letra y caracteres alfanuméricos)
- Valores límite: "", "abcdefg" (falla longitud) y "a", "abcdef" (valores dentro del rango)
- Validación carácter: "1abc" (primer caracter invalido)
- Validación letras y números: "ab\$12" (invalido) y "abc12" (valido)

2. BDD: Proceso de Retiro en Cajero Automático (Caso ATM)

- Utilizando la lógica del simulador de Cajero Automático (ATM) de la sesión anterior, aplicaremos BDD (Behavior-Driven Development) para validar que el sistema se comporte correctamente ante errores del usuario.
- Deben definir y probar el comportamiento del cajero cuando un usuario intenta retirar más dinero del que tiene disponible. Según Myers, este es un caso de prueba de "error" necesario para garantizar la robustez del software.
- Tareas a realizar:
 - Redactar la especificación (Gherkin) creando un archivo llamado atm.feature donde expliques el fallo usando el formato:
 - Dado que mi saldo es 100.
 - Cuando intento retirar 150.

- Entonces debo ver un mensaje de "Fondos Insuficientes".
- Implementar la prueba, deben traducir esos pasos a código Python para verificar que el simulador realmente bloquee la transacción.
- Entregables: atm.feature y test_atm.py.

Feature: Retiro de dinero en ATM

Scenario: Intento de retiro con fondos insuficientes

Given que mi saldo es 100

When intento retirar 150

Then debo ver un mensaje de "Fondos Insuficientes"

Scenario: Retiro exitoso

Given que mi saldo es 200

When intento retirar 50

Then el saldo final debe ser 150

Scenario: Retiro de saldo exacto

Given que mi saldo es 100

When intento retirar 100

Then el saldo final debe ser 0

Scenario: Retiro de monto cero

Given que mi saldo es 100

When intento retirar 0

Then el saldo final debe ser 100

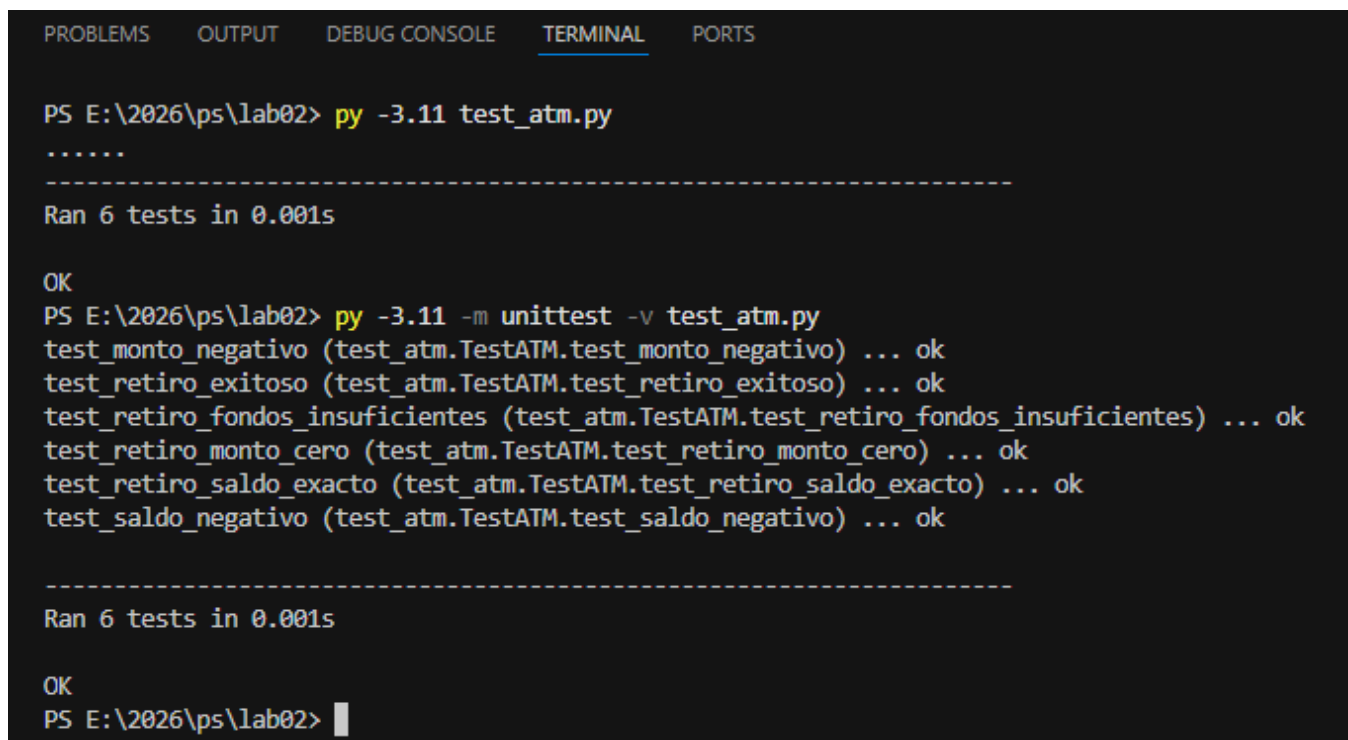
Código: test_atm.py

```
1  import unittest
2
3  def retirar(saldo, monto):
4      # Se valida que el monto ingresado no sea negativo,
5      # ya que no tiene sentido retirar una cantidad menor a cero
6      if monto < 0:
7          raise ValueError("El monto no puede ser negativo")
8
9      # Se verifica que el saldo inicial tampoco sea negativo,
10     # porque representaría un estado inválido del sistema
11     if saldo < 0:
12         raise ValueError("El saldo no puede ser negativo")
13
14     # Si el monto solicitado supera el saldo disponible,
15     # la operación debe ser rechazada mostrando un mensaje
16     if monto > saldo:
17         return "Fondos Insuficientes"
18
19     # Si todo es correcto, se realiza el retiro y se devuelve el nuevo saldo
20     return saldo - monto
21
```

```
22
23 class TestATM(unittest.TestCase):
24
25     # Caso principal basado en BDD:
26     # Se comprueba que el sistema bloquee retiros cuando no hay suficiente saldo
27     def test_retiro_fondos_insuficientes(self):
28         saldo = 100
29         monto = 150
30         resultado = retirar(saldo, monto)
31
32         self.assertEqual(resultado, "Fondos Insuficientes",
33             "Debe bloquear retiros mayores al saldo")
34
35     # Caso normal:
36     # Se verifica que un retiro valido reduzca correctamente el saldo
37     def test_retiro_exitoso(self):
38         saldo = 200
39         monto = 50
40         resultado = retirar(saldo, monto)
41
42         self.assertEqual(resultado, 150,
43             "Debe permitir retiro si hay saldo suficiente")
44
45     # Caso límite:
46     # Se evalúa el comportamiento cuando el usuario retira todo su saldo
47     def test_retiro_saldo_exacto(self):
48         saldo = 100
49         monto = 100
50         resultado = retirar(saldo, monto)
51
52         self.assertEqual(resultado, 0,
53             "Debe permitir retirar todo el saldo")
54
55     # Caso especial:
56     # Se comprueba que retirar 0 no altere el saldo
57     def test_retiro_monto_cero(self):
58         saldo = 100
59         monto = 0
60         resultado = retirar(saldo, monto)
61
62         self.assertEqual(resultado, 100,
63             "Retirar 0 no debe cambiar el saldo")
64
65     # Caso de robustez:
66     # Se valida que el sistema lance un error si el monto es negativo
67     def test_monto_negativo(self):
68         with self.assertRaises(ValueError):
69             retirar(100, -10)
```

```
70
71     # Caso de robustez:
72     # Se verifica que no se permita trabajar con un saldo inválido (negativo)
73     def test_saldo_negativo(self):
74         with self.assertRaises(ValueError):
75             retirar(-100, 50)
76
77
78 if __name__ == "__main__":
79     unittest.main()
```

Validación de Resultados: Ejecución de test_atm.py



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS E:\2026\ps\lab02> py -3.11 test_atm.py
.....
-----
Ran 6 tests in 0.001s

OK
PS E:\2026\ps\lab02> py -3.11 -m unittest -v test_atm.py
test_monto_negativo (test_atm.TestATM.test_monto_negativo) ... ok
test_retiro_exitoso (test_atm.TestATM.test_retiro_exitoso) ... ok
test_retiro_fondos_insuficientes (test_atm.TestATM.test_retiro_fondos_insuficientes) ... ok
test_retiro_monto_cero (test_atm.TestATM.test_retiro_monto_cero) ... ok
test_retiro_saldo_exacto (test_atm.TestATM.test_retiro_saldo_exacto) ... ok
test_saldo_negativo (test_atm.TestATM.test_saldo_negativo) ... ok

-----
Ran 6 tests in 0.001s

OK
PS E:\2026\ps\lab02> █
```

3. Validador de Calificaciones

- Se requiere una función evaluar_rendimiento(nota) que reciba un número entero entre 0 y 20.
- Requisitos:
 - o 0 - 10: "Insuficiente"
 - o 11 - 15: "Regular"
 - o 16 - 20: "Excelente"
- El estudiante debe aplicar TDD para manejar los rangos y lanzar una excepción (ValueError) si la nota está fuera del rango 0-20.
- Identificar clases de equivalencia válidas e inválidas.

Casos de Prueba

Primero, definimos categorías de casos de prueba como control para evaluar el output del programa. Se añadieron pruebas de robustez para entradas no numéricas siguiendo la técnica de error guessing, con el objetivo de garantizar que la función maneje correctamente entradas inválidas:

Clase	Salida
Nota insuficiente (Rango 0-10)	Insuficiente
Nota regular (Rango 11-15)	Regular
Nota excelente (Rango 16-20)	Excelente
Nota fuera de rango (<0, >20)	Error: Nota fuera de rango
Tipo de dato no entero	Error: Dato no entero

Fase RED

Escribimos las pruebas antes de implementar la función en el archivo **test_calificaciones.py**:

Python

```
import pytest
from calificaciones import evaluar_rendimiento

def test_insuficiente():
    assert evaluar_rendimiento(8) == "Insuficiente"

def test_regular():
    assert evaluar_rendimiento(13) == "Regular"

def test_excelente():
    assert evaluar_rendimiento(18) == "Excelente"

def test_valores_limite():
    assert evaluar_rendimiento(0) == "Insuficiente"
    assert evaluar_rendimiento(10) == "Insuficiente"
    assert evaluar_rendimiento(11) == "Regular"
    assert evaluar_rendimiento(15) == "Regular"
    assert evaluar_rendimiento(16) == "Excelente"
    assert evaluar_rendimiento(20) == "Excelente"

def test_fuera_de_rango():
    with pytest.raises(ValueError):
        evaluar_rendimiento(-1)

    with pytest.raises(ValueError):
        evaluar_rendimiento(21)
```

```
def test_tipo_invalido():  
  
    with pytest.raises(TypeError):  
        evaluar_rendimiento("15")  
  
    with pytest.raises(TypeError):  
        evaluar_rendimiento(None)  
  
    with pytest.raises(TypeError):  
        evaluar_rendimiento(15.5)
```

Por supuesto, al no estar implementada la función aún, falla:

```
PS D:\PruSofLab\02> python -m pytest  
===== test session starts =====  
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0  
rootdir: D:\PruSofLab\02  
collected 0 items / 1 error  
  
===== ERRORS =====  
ERROR collecting test_calificaciones.py  
ImportError while importing test module 'D:\PruSofLab\02\test_calificaciones.py'.  
Hint: make sure your test modules/packages have valid Python names.  
Traceback:  
C:\Users\Hydro\AppData\Local\Python\pythoncore-3.14-64\Lib\importlib\_init_.py:88: in import_module  
    return _bootstrap_gcd_import(name[level:], package, level)  
test_calificaciones.py:2: in <module>  
    from calificaciones import evaluar_rendimiento  
E   ModuleNotFoundError: No module named 'calificaciones'  
===== short test summary info =====  
ERROR test_calificaciones.py  
!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!  
===== 1 error in 0.19s =====  
PS D:\PruSofLab\02>
```

Fase GREEN

Ahora, implementamos el código mínimo necesario para que las pruebas pasen en el archivo **calificaciones.py**:

```
Python  
def evaluar_rendimiento(nota):  
  
    if not isinstance(nota, int):  
        raise TypeError("Error: Dato no entero")  
  
    if nota < 0 or nota > 20:  
        raise ValueError("Error: Nota fuera de rango")  
  
    if nota <= 10:  
        return "Insuficiente"  
    elif nota <= 15:  
        return "Regular"
```


	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

```
else:
    return "Excelente"
```

Al ejecutar, vemos que el programa supera todas las pruebas definidas en el archivo de test:

```
PS D:\PruSofLab\02> python -m pytest
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0
rootdir: D:\PruSofLab\02
collected 6 items

test_calificaciones.py ..... [100%]

===== 6 passed in 0.01s =====
PS D:\PruSofLab\02>
```

Fase REFACTOR

Revisamos el código para mejorar su claridad sin cambiar el comportamiento. En este caso, solo mejoramos la legibilidad sin cambiar la estructura, pues el problema es bastante corto:

```
Python
def evaluar_rendimiento(nota):

    # Validación de tipo
    if not isinstance(nota, int):
        raise TypeError("La nota debe ser un número entero")

    # Validación de rango
    if not 0 <= nota <= 20:
        raise ValueError("La nota debe estar entre 0 y 20")

    # Clasificación del rendimiento
    if nota <= 10:
        return "Insuficiente"

    if nota <= 15:
        return "Regular"

    return "Excelente"
```

Como vemos, no cambia el comportamiento del programa:

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

```
PS D:\PruSofLab\02> python -m pytest
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0
rootdir: D:\PruSofLab\02
collected 6 items

test_calificaciones.py ..... [100%]

===== 6 passed in 0.01s =====
PS D:\PruSofLab\02> |
```

4. Calculadora de Reembolsos

Deben crear un sistema que calcule cuánto dinero se le devuelve a un cliente cuando cancela una reserva de hotel, aplicando reglas especiales para clientes VIP.

Reglas de Negocio:

- Cancelación con mucha antelación (> 72 horas): Se devuelve el 100%.
- Cancelación con antelación media (24 a 72 horas): Se devuelve el 50%.
- Cancelación de última hora (< 24 horas): Se devuelve el 0%.
- Regla VIP: Si el cliente es VIP, siempre se le devuelve al menos el 50%, sin importar cuándo cancele.

Tareas del Estudiante:

1. Diseño BDD (Gherkin): Escriban un escenario donde un cliente VIP cancela con solo 2 horas de antelación y recibe su 50% de reembolso.
2. Pruebas de Robustez (TDD): Creen pruebas que verifiquen:
 - o Los valores límite (¿qué pasa exactamente a las 24 y 72 horas?).
 - o Protección contra errores: El sistema debe lanzar un error si el monto de la reserva es negativo.
3. Implementación: Programa la función `calcular_reembolso(monto, horas, es_vip)` asegurándote de que la regla VIP tenga prioridad sobre la regla de última hora.

Entregables:

- Archivo de texto con el escenario Gherkin y Código de las funciones y sus pruebas ejecutadas con éxito

Se realizó el diseño BDD(Gherkin) para escenarios con clientes no VIP y clientes VIP donde se especifica el escenario donde un cliente VIP cancela solo con 2 horas de antelación y recibe 50% de su reembolso.

Escenario: Cliente No VIP cancela su reserva con mucha antelación

- Dado que el cliente no es VIP y tiene una reserva de 200 soles
- Cuando cancela su reserva 80 horas antes de la fecha
- Entonces debe recibir el 100% de su reembolso, es decir, 200 soles

Escenario: Cliente No VIP cancela su reserva con antelación media

- Dado que el cliente no es VIP y tiene una reserva de 120 soles
- Cuando cancela su reserva 30 horas antes de la fecha

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

- Entonces debe recibir el 50% de su reembolso, es decir, 60 soles

Escenario: Cliente No VIP cancela su reserva a última hora

- Dado que el cliente no es VIP y tiene una reserva de 300 soles
- Cuando cancela su reserva 9 horas antes de la fecha
- Entonces no debe recibir reembolso, es decir, 0 soles

Escenario: Cliente VIP cancela su reserva con mucha antelación

- Dado que el cliente es VIP y tiene una reserva de 400 soles
- Cuando cancela su reserva 92 horas antes de la fecha
- Entonces debe recibir el 100% de su reembolso, es decir, 400 soles

Escenario: Cliente VIP cancela su reserva con antelación media

- Dado que el cliente es VIP y tiene una reserva de 250 soles
- Cuando cancela su reserva 44 horas antes de la fecha
- Entonces debe recibir el 50% de su reembolso, es decir, 125 soles

Escenario: Cliente VIP cancela su reserva a última hora

- Dado que el cliente es VIP y tiene una reserva de 500 soles
- Cuando cancela su reserva 2 horas antes de la fecha
- Entonces debe recibir al menos el 50% de su reembolso, es decir, 250 soles

Paso 1: Fase RED

Se hace uso de assert y pytest para las pruebas en test_reembolso.py

Dichas pruebas fallan porque aún no existe la función calcular_reembolso en [reembolso.py](#).

test_reembolso.py

Python

```
from reembolso import calcular_reembolso
```

```
def test_reembolso_noVip():
```

```
    assert calcular_reembolso(200, 80, False) == 200
```

```
    assert calcular_reembolso(120, 30, False) == 60
```

```
    assert calcular_reembolso(300, 9, False) == 0
```

```
def test_reembolso_vip():
```

```
    assert calcular_reembolso(400, 92, True) == 400
```

```
    assert calcular_reembolso(250, 44, True) == 125
```

```
    assert calcular_reembolso(500, 2, True) == 250
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

```
PS C:\Users\OFICINA\Documents\LabPS\Lab02> py -m pytest
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\OFICINA\Documents\LabPS\Lab02
collected 0 items / 1 error

===== ERRORS =====
_____ ERROR collecting test_reembolso.py _____
ImportError while importing test module 'C:\Users\OFICINA\Documents\LabPS\Lab02\test_reembolso.py'
.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
..\..\..\AppData\Local\Programs\Python\Python314\Lib\importlib\__init__.py:88: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
test_reembolso.py:1: in <module>
    from reembolso import calcular_reembolso
E   ImportError: cannot import name 'calcular_reembolso' from 'reembolso' (C:\Users\OFICINA\Documents\LabPS\Lab02\reembolso.py)
===== short test summary info =====
ERROR test_reembolso.py
!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!
===== 1 error in 0.36s =====
PS C:\Users\OFICINA\Documents\LabPS\Lab02>
```

Paso 2: Fase GREEN

Se crea el código mínimo para que se pasen las pruebas.

reembolso.py

Python

```
def calcular_reembolso(monto, horas, es_vip):
    if es_vip:
        if horas > 72:
            return monto
        elif horas < 72 and horas > 24:
            return monto * 0.5
        else:
            return monto * 0.5

    else:
        if horas > 72:
            return monto
        elif horas < 72 and horas > 24:
            return monto * 0.5
        else:
            return 0
```

```
PS C:\Users\OFICINA\Documents\LabPS\Lab02> py -m pytest
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\OFICINA\Documents\LabPS\Lab02
collected 2 items

test_reembolso.py .. [100%]

===== 2 passed in 0.03s =====
PS C:\Users\OFICINA\Documents\LabPS\Lab02>
```

Paso 3: Fase REFACTOR

Se ajusta y corrige la lógica para cumplir con las especificaciones, se añade robustez al código teniendo en cuenta el análisis de valores, límite de Myers, y en caso el monto sea negativo.

Python

```
def calcular_reembolso(monto, horas, es_vip):
    if monto < 0:
        raise ValueError("Monto no válido")

    if horas > 72:
        return monto
    elif horas <= 72 and horas >= 24:
        return monto * 0.5
    else:
        if es_vip:
            return monto * 0.5
        else:
            return 0
```

```
PS C:\Users\OFICINA\Documents\LabPS\Lab02> py -m pytest
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\OFICINA\Documents\LabPS\Lab02
collected 2 items

test_reembolso.py .. [100%]

===== 2 passed in 0.06s =====
PS C:\Users\OFICINA\Documents\LabPS\Lab02>
```

Se ajustan los parámetros de entrada en las pruebas para el análisis de datos teniendo en cuenta el límite de valores de Myers y la robustez en caso de monto negativo.

Python

`import pytest``from reembolso import calcular_reembolso``def test_reembolso_noVip():` `"""Pruebas para clientes no VIP"""` `#Reembolso completo` `assert calcular_reembolso(200, 73, False) == 200` `#Reembolso parcial` `assert calcular_reembolso(120, 72, False) == 60` `assert calcular_reembolso(120, 24, False) == 60` `#Reembolso cero` `assert calcular_reembolso(300, 23, False) == 0` `assert calcular_reembolso(300, 0, False) == 0``def test_reembolso_vip():` `"""Pruebas para clientes VIP"""` `#Reembolso completo` `assert calcular_reembolso(400, 73, True) == 400` `#Reembolso parcial` `assert calcular_reembolso(250, 72, True) == 125` `assert calcular_reembolso(250, 24, True) == 125` `#Reembolso parcial para VIP` `assert calcular_reembolso(500, 23, True) == 250` `assert calcular_reembolso(500, 0, True) == 250``def test_reembolso_monto_negativo():` `with pytest.raises(ValueError):` `calcular_reembolso(-100, 50, False)`

Vista en detalle de pytest

```
PS C:\Users\OFICINA\Documents\LabPS\Lab02> py -m pytest -v
===== test session starts =====
platform win32 -- Python 3.14.4, pytest-9.0.3, pluggy-1.6.0 -- C:\Users\OFICINA\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\OFICINA\Documents\LabPS\Lab02
collected 3 items

test_reembolso.py::test_reembolso_noVip PASSED [ 33%]
test_reembolso.py::test_reembolso_vip PASSED [ 66%]
test_reembolso.py::test_reembolso_monto_negativo PASSED [100%]

===== 3 passed in 0.03s =====
PS C:\Users\OFICINA\Documents\LabPS\Lab02> |
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15

II. CUESTIONARIO

1. ¿Por qué en TDD se considera que "la prueba es el primer cliente del código"?

En el Desarrollo Orientado a Pruebas (TDD), las pruebas se escriben antes de implementar el código. Esto significa que la primera entidad que interactúa con el sistema no es el usuario final, sino el caso de prueba, el cual define el comportamiento esperado desde el inicio [7].

La prueba actúa como una especificación ejecutable de los requisitos, guiando al desarrollador a implementar únicamente lo necesario para satisfacer dichas condiciones. Por ello, se considera que la prueba es el primer cliente del código, ya que valida inicialmente su correcto funcionamiento [6].

2. Explique cómo el enfoque BDD ayuda a evitar errores de comunicación entre el Product Owner y el equipo de desarrollo.

El enfoque de Desarrollo Guiado por Comportamiento (BDD) contribuye a reducir los malentendidos entre el Product Owner y el equipo de desarrollo al centrarse en describir el comportamiento esperado del sistema en un lenguaje claro y accesible para todos. En lugar de utilizar términos técnicos complejos, BDD promueve el uso de ejemplos concretos y comprensibles que facilitan que ambas partes compartan una misma visión del producto.[5]

Además, BDD fomenta la colaboración constante entre los diferentes actores del proyecto, incluyendo desarrolladores, testers y el Product Owner. A través de la definición conjunta de escenarios y criterios de aceptación, se logra alinear expectativas desde el inicio, evitando interpretaciones erróneas que podrían derivar en funcionalidades incorrectas o incompletas.[8]

Por último, al documentar los requisitos en forma de escenarios verificables, BDD permite validar continuamente que el sistema cumple con lo esperado. [10] Esto no solo ayuda a detectar errores de forma temprana, sino que también asegura que el producto final responda realmente a las necesidades del negocio, reduciendo retrabajos y mejorando la comunicación durante todo el proceso de desarrollo.

3. Según Myers, ¿qué es el "análisis de valor límite" y por qué es más efectivo que probar solo valores aleatorios?

Según Myers en el "análisis de valor límite" consiste en evaluar los valores extremos dentro de un rango, es decir, los valores mínimos, máximos y cercanos a los límites. Según Myers los errores tienden a ocurrir en sus límites de entrada donde pueden existir problemas en las condiciones lógicas o validaciones. [6] Por ejemplo:

Si un sistema acepta valores entre 1 y 6, no únicamente se debe probar valores intermedios como 2 o 3, es más efectivo probar los valores límites y sus alrededores como [0,1,6,7] ya que estos representan

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 16

los límites y su alrededor.

Esta técnica es mucho más efectiva que únicamente probar con valores aleatorios porque cubre casos críticos donde es bastante más probable que ocurra. En cambio las aleatorias no aseguran puntos críticos lo que pueden dejar puntos clave donde no se evaluaron.

4. ¿Cuál es el propósito del paso Refactor en el ciclo de TDD y qué riesgo se corre si se omite?

El paso Refactor en el ciclo de TDD tiene como objetivo mejorar la estructura interna del código sin modificar su comportamiento externo.[7]

Después de que las pruebas pasan en la fase Green, el código suele ser una implementación mínima diseñada únicamente para satisfacer los casos de prueba. En la fase de refactorización se busca mejorar la legibilidad del código, eliminar duplicación, simplificar la lógica y aplicar buenas prácticas de diseño.

Las pruebas automatizadas garantizan que, durante este proceso de mejora, el comportamiento del sistema se mantenga correcto.[10]

Si se omite la fase de refactorización, el principal riesgo es que el código se vuelva difícil de mantener y escalar, acumulando complejidad innecesaria y posibles errores futuros. Con el tiempo esto puede generar deuda técnica, haciendo que el sistema sea más difícil de modificar o extender.

5. Explique la importancia de definir el resultado esperado antes de la ejecución de una prueba.

Definir el resultado esperado antes de ejecutar una prueba es fundamental porque establece un criterio claro para determinar si el comportamiento del sistema es correcto o no. Sin esta referencia previa, sería difícil evaluar objetivamente los resultados obtenidos y saber si la funcionalidad cumple con los requisitos.[3]

Asimismo, anticipar el resultado permite diseñar pruebas más precisas y enfocadas, ya que se tiene claridad sobre qué se quiere validar. [5] Esto ayuda a evitar ambigüedades y reduce la posibilidad de interpretaciones distintas entre los miembros del equipo, mejorando la calidad del proceso de pruebas.

Finalmente, contar con resultados esperados bien definidos facilita la detección temprana de errores y agiliza la toma de decisiones. Cuando los resultados reales no coinciden con lo esperado, se puede identificar rápidamente la existencia de fallos, lo que contribuye a corregirlos de manera oportuna y eficiente.[2]

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 17

III. CONCLUSIONES

- En esta práctica se aplicó el enfoque de Desarrollo Orientado a Pruebas (TDD), donde primero se escriben las pruebas antes de implementar el código. Esto permitió definir claramente el comportamiento esperado del sistema y asegurar que las funciones cumplan con los requisitos planteados.
- El ciclo Red–Green–Refactor ayudó a desarrollar el software de manera ordenada. Primero se identificaron los casos de prueba que debían fallar, luego se implementó el código mínimo para que pasen las pruebas, y finalmente se mejoró la estructura del código sin cambiar su comportamiento.
- El uso de pruebas permitió verificar distintos escenarios, incluyendo casos normales, valores límite y situaciones de error. Esto contribuye a aumentar la confiabilidad del sistema y facilita la detección temprana de fallos.
- Además, el uso de BDD permitió describir el comportamiento del sistema mediante escenarios comprensibles, lo que ayuda a mejorar la comunicación entre los miembros del equipo y a entender mejor los requisitos del problema.
- Se pudo comprobar que combinar TDD y BDD permite desarrollar software de forma más segura, organizada y con mayor calidad, ya que el comportamiento del sistema se valida continuamente mediante pruebas automatizadas.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Amazon Web Services (AWS). What is Debugging? <https://aws.amazon.com/es/what-is-debugging/>
- [2] IBM. ¿Qué es la depuración (debugging)? <https://www.ibm.com/mx-es/think/topics/debugging>
- [3] ISTQB. (s.f.). What is software testing? <https://www.istqb.org/>
- [4] IBM. (s.f.). Software testing. <https://www.ibm.com/topics/software-testing>
- [5] ISTQB. (s.f.). Certified Tester Foundation Level Syllabus. <https://www.istqb.org/>
- [6] Myers, G. J., Sandler, C., & Badgett, T. (2011). The art of software testing (3rd ed.). John Wiley & Sons.
- [7] Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed.). McGraw-Hill.
- [8] Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed., pp. 459–480). McGraw-Hill.
- [9] Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner's approach (9th ed., pp. 418–430). McGraw-Hill.
- [10] Sommerville, I. (2016). Software engineering (10th ed.). Pearson.